

TP 4 : TUBES DE COMMUNICATION UNIX
Systèmes d'Exploitation – SR1

07-10-2026

Exercice 1

Écrire un programme C qui utilise un tube de communication entre deux processus de la manière suivante :

- créer un tube de communication ;
- créer un processus fils ;
- le processus père envoie au fils un entier toutes les secondes, en l'incrémentant de 1 à N ;
- puis le père ferme le tube en écriture ;
- le fils lit et affiche la valeur de l'entier tant qu'il peut lire dans le tube ;
- puis il s'arrête avec un code de retour égal à 1 ;
- le père attend la fin du fils et affiche le code de retour de celui-ci avant de se terminer.

Par exemple, si N = 4 :

```
>TP_4.1
[Fils] 1
[Fils] 2
[Fils] 3
[Fils] 4
[Père] : Mon fils 10382 est terminé avec le code 1.
```

Exercice 2

Écrire un programme C qui simule le remplissage d'un bassin avec deux tubes de communication entre 2 processus :

- créer un tube de communication **Remplir** et un tube **Etat** ;
- créer un processus fils ;
- le processus père met le tube **Etat** en mode non bloquant en lecture (*cf. exemple ci-dessous*) ;
- le processus père écrit au fils dans le tube **Remplir** un entier égal à 1 toutes les secondes pendant N fois ;
- à chaque itération, le père tente de lire un entier dans le tube **Etat** (rappel **Etat** est en mode non bloquant). Si la lecture marche, le père écrit un entier égal à -1 dans le tube **Remplir**, puis continue sa boucle ;
- le fils lit un entier dans le tube **Remplir**, en boucle tant qu'il peut. Et :
 - si l'entier lu est positif, il l'ajoute à une variable **bassin**. Si **bassin** est supérieur à une constante **MAX_BASSIN** alors le fils écrit l'entier **bassin** dans le tube **Etat** ;
 - s'il est négatif, **bassin** est mis à zéro (le bassin est vidangé) ;
- quand le fils ne peut plus lire il s'arrête avec un code de retour égal à 1 ;
- le père attend la fin du fils et affiche le code de retour de celui-ci avant de se terminer.

```
#include <fcntl.h>
```

```
...
```

```
int f_flags;                /* mode non bloquant pour le tube Etat dans le Père */
f_flags = fcntl(Etat[0], F_GETFL); /* Récupération des flags */
f_flags |= O_NONBLOCK;       /* Positionnement du flag de non blocage */
fcntl(Etat[0], F_SETFL, f_flags); /* Mis à jour des flags */
```

On peut faire afficher le remplissage du bassin par le fils avec `printf("*"); fflush(stdout);` à chaque fois qu'il incrémente `bassin` et `printf("\n");` quand il le vidange.

Par exemple, si `N = 10` et `MAX_BASSIN = 5` :

>TP_4.2

[Père] : Mon fils 10462 est terminé avec le code 1.

Exercice 3

Écrire un programme `premier`, qui prend en paramètre une liste de nombres entiers naturels et affiche la liste de ces nombres qui sont premiers. La syntaxe d'appel du programme est la suivante :

`premier n1 n2...` où `n1 n2...` est une liste d'entiers naturels.

Le programme doit créer `N` processus fils (par exemple `N=4`) qui vont s'exécuter en parallèle. Le processus père convertit chaque paramètre en entier (`unsigned long long int` avec la fonction `strtoull` – cf. `man strtoull`¹) et l'écrit dans un tube, puis ferme le tube et attend la fin de tous ses fils. Chaque processus fils doit :

- lire en boucle un entier dans le tube ;
- vérifier si cet entier est premier en utilisant la fonction (*qui est fournie*) :
`int est_premier (unsigned long long int n)`
 qui renvoie 1 si l'entier `n` est premier et 0 sinon.
- afficher son numéro suivi du nombre ainsi que `premier` si celui-ci est premier (rien de plus sinon) ;
- incrémenter un compteur de nombres premiers si le nombre est premier ;
- boucler ainsi tant que le tube est ouvert ;
- se terminer avec comme code de retour le nombre d'entiers premiers trouvés.

Le processus père attend la fin de tous ses fils et affiche le nombre total de nombres premiers.

Le programme doit vérifier la validité des paramètres et afficher les éventuels messages d'erreur.

Pour l'affichage d'un `unsigned long long int` on utilise `printf("%llu", ...)`

Par exemple, si `N = 4` :

>premier 0 5769068314253 1 2 18446744073709551557 3 4 5 6 7 8 9 10 11

```
[Fils 2]          0
[Fils 0]          1
[Fils 0]          3 est premier
[Fils 0]          4
[Fils 0]          5 est premier
[Fils 3]          2 est premier
[Fils 0]          6
[Fils 0]          7 est premier
[Fils 0]          8
[Fils 0]          9
[Fils 3]          10
[Fils 3]          11 est premier
[Fils 1]          5769068314253 est premier
[Fils 2]18446744073709551557 est premier
Il y a 7 nombres premiers.
```

1. Attention, `strtoull` ne fait pas d'erreur si le paramètre contient un entier négatif mais renvoie un entier naturel!